Az `organize_and_rename_file` Függvény Sorról Sorra Elemzése

Ez a függvény felelős az automatizáció **fizikai fájlkezelési és archiválási** lépéséért. Miután az OCR és a DALI-összerendelés kinyerte az adatokat, ez a kód hozza létre a célmappákat a háttértáron, nevezi át a PDF-eket a megadott szabvány szerint, és másolja át őket a végleges helyükre.

---

1. A Teljes Függvénykód

```
def organize_and_rename_file(pdf_path: Path, metadata: dict, target_base: Path):
    """Létrehozza a célmappát és átnevezi a fájlt."""
    date = metadata.get("Dátum", "NA_D")
    invoice = metadata.get("Invoice from DALI", "NO_INV")
    shipment = metadata.get("Shipment No.", "NA_S")
    year = date.split("-")[0] if date.startswith("20") else "Vegyes"

    destination_dir = target_base / year
    destination_dir.mkdir(parents=True, exist_ok=True)

    clean_filename = re.sub(r'[\\/*?:"<>|]', "_", f"{date}_{invoice}_{shipment}.pdf")
    dest_path = destination_dir / clean_filename
    shutil.copy2(pdf_path, dest_path)
    return str(dest_path)
```

---

2. Sorról Sorra Történő Szintaktikai és Logikai Elemzés

1. Sor: Függvényfejléc és Bemenő Paraméterek

```
def organize_and_rename_file(pdf_path: Path, metadata: dict, target_base: Path):
```

- **Szintaktika:**
  - `def`: Függvénydefiníció.
  - `pdf_path: Path`: Az eredeti forrásfájl elérési útja (pl. `Path(r"C:\...\Proba PoD automata\50643367_2026.04.27.pdf")`).
  - `metadata: dict`: A `parse_metadata` függvény által előállított szótár, ami tartalmazza a kinyert Dátumot, Számlaszámot, Shipmentet stb.
  - `target_base: Path`: A rendezett fájlok fő gyökérmappája (`Rendezett POD-k`).
- **Logika:** A függvénynek pontosan tudnia kell, **mit** másoljon (`pdf_path`), **milyen adatok alapján** nevezze át (`metadata`), és **hová** mentse (`target_base`).

2–4. Sor: Adatkinyerés Biztonsági Tartalékértékekkel (`.get()`)

```
date = metadata.get("Dátum", "NA_D")
invoice = metadata.get("Invoice from DALI", "NO_INV")
shipment = metadata.get("Shipment No.", "NA_S")
```

- **Szintaktika:**
  - `szótár.get(kulcs, alapértelmezett_érték)`: A Python szótárak biztonságos lekérdező metódusa.
  - Ha a kulcs létezik a szótárban, visszaadja az értékét.
  - Ha a kulcs nem létezik (vagy `None`), **nem omlik össze a program `KeyError` hibával**, hanem a második paraméterben megadott szöveget adja vissza.
- **Logika:**
  - Ha sikerült kinyerni a dátumot, az lesz a `date`. Ha nem (vagy hiányzik), a fájlnévben `NA_D` (*Not Available Date*) jelenik meg.
  - Ha a DALI táblából megtaláltuk a számlaszámot, az lesz az `invoice`. Ha nem szerepel benne, `NO_INV` (*No Invoice*) kerül a névbe.
  - Ha megvan a Shipment szám, az kerül be, ellenkező esetben `NA_S` (*No Shipment*).

---

5. Sor: Intelligens Évszám-alapú Mappaválasztás (Ternary Operator)

```
year = date.split("-")[0] if date.startswith("20") else "Vegyes"
```

- **Szintaktika:**
  - Ez egy egysoros feltételes értékadás (*Inline If / Ternary Operator*).
  - `date.startswith("20")`: Megvizsgálja, hogy a dátum szövege `20`-szal kezdődik-e (pl. `2025` vagy `2026`).
  - `date.split("-")[0]`: A kötőjelek mentén darabolja a dátumot (`2026-04-27` ➔ `['2026', '04', '27']`), és a `[0]` indexszel kiválasztja a legelső elemet, vagyis magát az évszámot (`2026`).
  - `else "Vegyes"`: Ha a dátum ismeretlen maradt (`Review Manually` vagy `NA_D`), akkor az évszám helyett a `"Vegyes"` mappanevet kapja.
- **Logika:** A program automatikusan szétválogatja a fájlokat: a 2025-ös dokumentumok a `2025`, a 2026-osak a `2026` almappába vándorolnak. A felismerhetetlen dátumú lapok nem vesznek el, hanem a `Vegyes` mappába gyűlnek kézi átnézésre.

6–7. Sor: Célmappa Létrehozása a Merevlemezen

```
destination_dir = target_base / year
destination_dir.mkdir(parents=True, exist_ok=True)
```

- **Szintaktika:**
  - `/` operátor a `pathlib.Path` objektumoknál: Platformfüggetlen útvonal-összefűzés. Automatikusan a megfelelő visszaperjelet (`\`) teszi ki Windows alatt: `C:\...\Rendezett POD-k` + `2026` ➔ `C:\...\Rendezett POD-k\2026`.
  - `.mkdir(...)`: Mappát hoz létre a fájlrendszerben.
    - `parents=True`: Ha a szülőmappa (`Rendezett POD-k`) még nem létezne, azt is automatikusan létrehozza.

        - `exist_ok=True`: **NAGYON FONTOS!** Ha a mappa már létezik a merevlemezen, nem dob hibát, hanem egyszerűen használja azt.
- **Logika:** Garantálja, hogy a célkönyvtár biztosan létezzen a fájl másolása előtt.

---

8. Sor: Fájlnév Generálás és Tiltott Karakterek Tisztítása

```
clean_filename = re.sub(r'[\\/*?:"<>|]', "_", f"{date}_{invoice}_{shipment}.pdf")
```

- **Szintaktika:**
    - `f"{date}_{invoice}_{shipment}.pdf"`: F-string formázás. Összefűzi a kívánt fájlnevet, pl.: `2026-04-27_50643367_2004812302.pdf`.
    - `re.sub(minta, mire, miben)`: Regex helyettesítés (*substitution*).
    - `r'[\\/*?:"<>|]'`: Karakterosztály `[...]`. A Windows fájlrendszer által **szigorúan tiltott** 9 írásjelet sorolja fel:
        - `\` (visszaperjel)
        - `/` (perjel)
        - `*` (csillag)
        - `?` (kérdőjel)
        - `:` (kettőspont)
        - `"` (idézőjel)
        - `<` és `>` (kacsacsőrök)
        - `|` (függőleges vonal)
    - `"_"`: Ha a DALI-ból vagy az OCR-ből véletlenül ilyen tiltott karakter kerülne a számlaszámba (pl. perjel `/`), azt azonnal aláhúzásjelre `_` cseréli.
- **Logika:** Megakadályozza, hogy a Windows `OSError: [Errno 22] Invalid argument` hibával leállítsa a programot a fájl mentésekor.

9–11. Sor: Másolás és Eredmény Visszaadása

```
dest_path = destination_dir / clean_filename
shutil.copy2(pdf_path, dest_path)
return str(dest_path)
```

- **Szintaktika:**
    - `dest_path = destination_dir / clean_filename`: A teljes cél-útvonal a megtisztított új fájlnévvel.
    - `shutil.copy2(forrás, cél)`:
        - Fizikailag átmásolja a fájlt a forrásmappából a célmappába.
        - A `copy2` (a sima `copy`-val szemben) **megőrzi az eredeti fájl metaadatait is** (pl. létrehozás és utolsó módosítás időbélyege).
    - `return str(dest_path)`: A célútvonalat szövegként (`string`) adja vissza a hívó `main()` függvénynek.
- **Logika:** A visszakapott elérési utat a `main()` azonnal beírja az Excel táblázat **"Új Fájl Útvonala"** oszlopába, így a riportban pontosan látszik, melyik dokumentum hová került átmentésre.

---

3. Hogyan Tudsz Manuálisan Belenyúlni a Működésbe?

1. Fájlnév-struktúra Megváltoztatása:

Ha a céges előírás szerint a számlaszámnak kell elöl állnia, vagy a CMR számot is bele akarod venni a fájlnévbe:

1. Vedd át a CMR számot is a metaadatból:

   ```
   cmr = metadata.get("CMR Szám", "NO_CMR")
   ```

2. Módosítsd az f-stringet a 8. sorban:

   ```
   # Új sorrend: Számlaszám_CMR_Dátum.pdf
   clean_filename = re.sub(r'[\\/*?:"<>|]', "_", f"{invoice}_{cmr}_{date}.pdf")
   ```

2. Almappák Struktúrájának Finomítása (pl. Hónapok szerinti bontás):

Ha nemcsak év, hanem **év és hónap** (`2026-04`) szerinti almappákat szeretnél:

```
# Év-Hónap mappa készítése (pl. '2026-04')
year_month = date[:7] if date.startswith("20") else "Vegyes"
destination_dir = target_base / year_month
```

3. Másolás Helyett Mozgatás:

Ha azt szeretnéd, hogy a sikeresen feldolgozott PDF-ek ne másolódjanak, hanem **végleg átkerüljenek** a forrásmappából a célmappába (így kiürítve a bejövő mappát):

- Cseréld ki a `shutil.copy2`-t erre:

  ```
  shutil.move(pdf_path, dest_path)
  ```